

GlicoCare - Specifiche per Class Diagram UML

Generato il: 07/07/2026 10:27

Contiene la definizione completa di Classi, Attributi e Metodi per la progettazione del diagramma (inclusi costruttori e metodi privati).

■ Modulo: main.py

Percorso: `main.py`

Nessuna classe pubblica definita in questo modulo.

■ Modulo: md2pdf.py

Percorso: `md2pdf.py`

Nessuna classe pubblica definita in questo modulo.

■ Modulo: user.py

Percorso: `src/user.py`

■■ Classe: CredenzialiNonValide

Descrizione: Eccezione sollevata quando le credenziali di accesso non sono valide.

Estende: Exception, BaseException

■ Attributi (Variabili di istanza):

Nome	Tipo suggerito	Descrizione
args	Any	
kwargs	Any	

■ Costruttore:

+ **CredenzialiNonValide(self, /, *args, **kwargs)**

Descrizione: Initialize self. See help(type(self)) for accurate signature.

■ Metodi (UML: + pubblico, - privato):

Nessun metodo (oltre al costruttore) definito.

■■ Classe: User

Descrizione: Rappresenta un utente del sistema e gestisce l'autenticazione. Args: username (str): Nome utente da autenticare. password (str): Password in chiaro da verificare. db_path (str, optional): Percorso del file del database SQLite. Default: "glicocare.db". Raises: CredenzialiNonValide: Se le credenziali fornite non sono corrette.

■ Attributi (Variabili di istanza):

Nome	Tipo suggerito	Descrizione
username	<class 'str'>	Nome utente da autenticare.
password	<class 'str'>	Password in chiaro da verificare.
db_path	<class 'str'>	Percorso del file del database SQLite.

■ Costruttore:

+ User(self, username: str, password: str, db_path: str = 'glicocare.db')

Descrizione: Initialize self. See help(type(self)) for accurate signature.

■ Metodi (UML: + pubblico, - privato):

■ - __str__

Descrizione: Restituisce una rappresentazione leggibile dell'utente.

Firma: __str__(self) -> str

Ritorna: str: Stringa riassuntiva dell'utente.

■ - _get_connection

Descrizione: Crea e restituisce una connessione al database.

Firma: _get_connection(self) -> sqlite3.Connection

Ritorna: sqlite3.Connection: Oggetto connessione al database.

■ - _hash_password

Descrizione: Calcola l'hash SHA-256 di una password.

Firma: _hash_password(password: str) -> str

Ritorna: password (str): Password in chiaro. str: Hash della password in formato esadecimale.

■ - _verifica_credenziali

Descrizione: Verifica le credenziali dell'utente nel database. Se la verifica ha successo, gli attributi `tipo` e `id\_ref` vengono popolati con i valori letti dal database.

Firma: _verifica_credenziali(self, password: str) -> bool

Ritorna: password (str): Password in chiaro da verificare. bool: True se le credenziali sono valide, False altrimenti.

■ + get_user_info

Descrizione: Restituisce un dizionario con le informazioni dell'utente autenticato.

Firma: get_user_info(self) -> dict

Ritorna: dict: Dizionario con chiavi 'username', 'tipo' e 'id_ref'.

■ + is_medico

Descrizione: Verifica se l'utente è un medico.

Firma: is_medico(self) -> bool

Ritorna: bool: True se l'utente è di tipo 'M', False altrimenti.

■ + is_paziente

Descrizione: Verifica se l'utente è un paziente.

Firma: is_paziente(self) -> bool

Ritorna: bool: True se l'utente è di tipo 'P', False altrimenti.

■ Modulo: graph_utils.py

Percorso: `src/graph\_utils.py`

Nessuna classe pubblica definita in questo modulo.

■ Modulo: medico.py

Percorso: `src/medico.py`

■■ Classe: Medico

Descrizione: Rappresenta un medico e fornisce metodi per gestire i suoi pazienti. Args:
id_ref (int): L'ID del medico (corrisponde a `id\_med` nella tabella MEDICO). db_path (str, optional): Percorso del file del database SQLite. Default: "database/glicocare.db".

■ Attributi (Variabili di istanza):

Nome	Tipo suggerito	Descrizione
id_ref	Any	L'ID del medico (corrisponde a `id_med` nella tabella MEDICO).
db_path	Any	Percorso del file del database SQLite.

■ Costruttore:

+ **Medico**(self, id_ref, db_path=None)

Descrizione: Initialize self. See help(type(self)) for accurate signature.

■ Metodi (UML: + pubblico, - privato):

■ - __str__

Descrizione: Return str(self).

Firma: __str__(self) -> str

■ - _aggiorna_gravita_paziente

Descrizione: Forza il ricalcolo della gravità per un paziente specifico.

Firma: _aggiorna_gravita_paziente(self, id_paz: int)

■ - _carica_dati

Descrizione: Carica i dati anagrafici del medico e la lista dei suoi pazienti dal DB.

Firma: _carica_dati(self)

■ + eliminaTerapiaDefinitiva

Descrizione: Cancella fisicamente una terapia dal database.

Firma: eliminaTerapiaDefinitiva(self, id_paz: int, farmaco: str)

■ + getCel

Firma: getCel(self) -> str

■ + getCf

Firma: getCf(self) -> str

■ + getCognome

Firma: getCognome(self) -> str

■ + getDataNascita

Firma: getDataNascita(self) -> str

■ + getEmail

Firma: getEmail(self) -> str

■ + getIdRef

Firma: getIdRef(self) -> int

■ + getIndirizzo

Firma: getIndirizzo(self) -> str

■ + getLuogoNascita

Firma: getLuogoNascita(self) -> str

■ + getNome

Firma: getNome(self) -> str

■ + getPazienteById

Firma: getPazienteById(self, id_paz: int)

■ + getPazienti

Firma: getPazienti(self) -> list

■ + getPazientiConDettaglio

Descrizione: Restituisce una lista di dizionari con i dettagli di ogni paziente, ordinati per gravità decrescente.

Firma: getPazientiConDettaglio(self) -> list

■ + getPazientIds

Firma: getPazientiIds(self) -> list

■ + getSesso

Firma: getSesso(self) -> str

■ + getStatistiche

Firma: getStatistiche(self) -> dict

■ + get_log_operazioni

Firma: get_log_operazioni(self, limite=50)

■ + interrompiTerapia

Descrizione: Imposta la data di fine di una terapia (senza cancellarla).

Firma: interrompiTerapia(self, id_paz: int, farmaco: str, data_fine: str = None)

■ + modificaTerapia

Descrizione: Modifica una terapia esistente. CASO A: Stesso farmaco -> UPDATE diretto (modifica dose, data_fine, ecc.) CASO B: Farmaco diverso -> Chiudi vecchia (data_fine=oggi) e crea nuova riga.

Firma: modificaTerapia(self, id_paz: int, vecchio_farmaco: str, nuovo_farmaco: str, assunzioniGiornaliere: int, quantita: str, data_inizio: str, data_fine: str = None, indicazioni: str = None)

■ + prescriviTerapia

Descrizione: Prescrive una nuova terapia (INSERT).

Firma: prescriviTerapia(self, id_paz: int, farmaco: str, assunzioniGiornaliere: int, quantita: str, data_inizio: str, data_fine: str = None, indicazioni: str = None)

■ + registra_operazione

Firma: registra_operazione(self, azione: str, tabella: str, id_record: str = None, dettaglio: str = None) -> None

■ Modulo: paziente.py

Percorso: `src/paziente.py`

■■ Classe: Paziente

Descrizione: Nessuna descrizione disponibile.

■ Attributi (Variabili di istanza):

Nome	Tipo suggerito	Descrizione
id_ref	Any	
db_path	Any	

■ Costruttore:

+ Paziente(self, id_ref, db_path=None)

Descrizione: Initialize self. See help(type(self)) for accurate signature.

■ Metodi (UML: + pubblico, - privato):

■ - __str__

Descrizione: Return str(self).

Firma: __str__(self) -> str

■ - _aggiorna_gravita_db

Descrizione: Ricalcola la gravità e la aggiorna nel database e nell'oggetto.

Firma: _aggiorna_gravita_db(self)

■ - _calcola_gravita

Descrizione: Calcola la gravità del paziente: - Ogni valore glicemico fuori soglia = 1 punto
- Ogni giorno con rilevazioni dove manca almeno un'assunzione (basato sulle terapie attive) = 1 punto

Firma: _calcola_gravita(self)

Ritorna: tuple: (punteggio_totale, punti_glicemia, punti_terapia)

■ - _carica_dati

Firma: _carica_dati(self)

■ - _ordina_rilevazioni

Descrizione: Ordina le rilevazioni per giorno decrescente e ora crescente.

Firma: _ordina_rilevazioni(self)

■ - _refresh_rilevazioni

Descrizione: Ricarica la lista delle rilevazioni dal database dopo una modifica.

Firma: _refresh_rilevazioni(self)

■ + aggiornaAnnotazione

Descrizione: Aggiorna l'annotazione clinica nel database e in memoria.

Firma: aggiornaAnnotazione(self, testo: str) -> None

■ + aggiornaAssunzione

Descrizione: Aggiorna un'assunzione esistente.

Firma: aggiornaAssunzione(self, vecchio_giorno: str, vecchia_ora: str, vecchio_farmaco: str, nuovo_giorno: str, nuova_ora: str, farmaco: str, quantita: str) -> None

■ + aggiornaRilevazioneGiornaliera

Descrizione: Aggiorna una rilevazione della glicemia esistente.

Firma: aggiornaRilevazioneGiornaliera(self, vecchio_giorno, vecchia_ora, nuovo_giorno, nuova_ora, glicemia, primaDopoPasto)

Eccezioni sollevate: ValueError: Se glicemia < 0 o primaDopoPasto non è 'P' o 'D'

■ + aggiornaSegnalazione

Descrizione: Aggiorna una segnalazione esistente.

Firma: aggiornaSegnalazione(self, vecchio_giorno: str, vecchia_ora: str, nuovo_giorno: str, nuova_ora: str, sintomo: str, terapia: str) -> None

■ + aggiungiAssunzione

Descrizione: Registra un'assunzione di un farmaco. Recupera automaticamente data_inizio dalla terapia attiva.

Firma: aggiungiAssunzione(self, giorno: str, ora: str, farmaco: str, quantita: str) -> None

■ + aggiungiRilevazioneGiornaliera

Descrizione: Aggiunge una nuova rilevazione della glicemia.

Firma: aggiungiRilevazioneGiornaliera(self, giorno: str, ora: str, glicemia: float, primaDopoPasto: str) -> None

Eccezioni sollevate: ValueError: Se glicemia < 0 o primaDopoPasto non è 'P' o 'D'

■ + aggiungiSegnalazione

Descrizione: Aggiunge una nuova segnalazione.

Firma: aggiungiSegnalazione(self, giorno: str, ora: str, sintomo: str, terapia: str = None) -> None

Nome	Tipo	Descrizione
giorno	str	Data della segnalazione nel formato 'YYYY-MM-DD'.

ora	str	Ora della segnalazione nel formato 'HH:MM'.
sintomo	str	Descrizione del sintomo.
terapia	str, optional	Nome del farmaco associato, se presente.

■ + eliminaAssunzione

Descrizione: Elimina un'assunzione esistente.

Firma: eliminaAssunzione(self, giorno: str, ora: str, farmaco: str) -> None

■ + eliminaRilevazioneGiornaliera

Descrizione: Elimina una rilevazione della glicemia.

Firma: eliminaRilevazioneGiornaliera(self, giorno: str, ora: str) -> None

■ + eliminaSegnalazione

Descrizione: Elimina una segnalazione esistente.

Firma: eliminaSegnalazione(self, giorno: str, ora: str) -> None

■ + getAnnotazione

Descrizione: Restituisce l'annotazione clinica del paziente.

Firma: getAnnotazione(self) -> str

■ + getAssunzioni

Firma: getAssunzioni(self) -> list

■ + getCel

Descrizione: Restituisce il numero di cellulare del paziente.

Firma: getCel(self) -> str

■ + getCf

Descrizione: Restituisce il codice fiscale del paziente.

Firma: getCf(self) -> str

■ + getCognome

Descrizione: Restituisce il cognome del paziente.

Firma: getCognome(self) -> str

■ + getDataNascita

Descrizione: Restituisce la data di nascita del paziente.

Firma: getDataNascita(self) -> str

■ + getEmail

Descrizione: Restituisce l'email del paziente.

Firma: getEmail(self) -> str

■ + getGravita

Descrizione: Restituisce il livello di gravità calcolato.

Firma: getGravita(self) -> int

■ + getIdRef

Descrizione: Restituisce l'ID di riferimento del paziente.

Firma: getIdRef(self) -> int

■ + getIndirizzo

Descrizione: Restituisce l'indirizzo del paziente.

Firma: getIndirizzo(self) -> str

■ + getLuogoNascita

Descrizione: Restituisce il luogo di nascita del paziente.

Firma: getLuogoNascita(self) -> str

■ + getMedicId

Descrizione: Restituisce l'ID del medico di riferimento.

Firma: getMedicoId(self) -> int

■ + getMedicoRiferimento

Descrizione: Recupera i dati anagrafici del medico di riferimento del paziente.

Firma: getMedicoRiferimento(self) -> tuple

Ritorna: tuple | None: Tupla nel formato (nome, cognome, email, cel) del medico, oppure None se non trovato.

■ + getNome

Descrizione: Restituisce il nome del paziente.

Firma: getNome(self) -> str

■ + getPuntiGlicemia

Descrizione: Restituisce i punti gravità legati alla glicemia fuori soglia.

Firma: getPuntiGlicemia(self) -> int

■ + getPuntiTerapia

Descrizione: Restituisce i punti gravità legati alla terapia non rispettata.

Firma: getPuntiTerapia(self) -> int

■ + getRilevazioni

Descrizione: Restituisce tutte le rilevazioni della glicemia del paziente.

Firma: getRilevazioni(self) -> list

Ritorna: list: Lista di tuple nel formato (giorno (str), ora (str), glicemia (float), primaDopoPasto (str)).

■ + getSegnalazioni

Descrizione: Recupera tutte le segnalazioni del paziente.

Firma: getSegnalazioni(self) -> list

Ritorna: list: Lista di tuple nel formato (giorno (str), ora (str), sintomo (str), terapia (str|None)).

■ + getSesso

Descrizione: Restituisce il sesso del paziente ('M' o 'F').

Firma: getSesso(self) -> str

■ + getTerapie

Descrizione: Restituisce le terapie farmacologiche ATTIVE del paziente.

Firma: `getTerapie(self) -> list`

Ritorna: list: Lista di tuple nel formato (farmaco (str), assunzioniGiornaliere (int), quantita (str), indicazioni (str), id_med (int), data_inizio (str)).

■ + `getTerapieByName`

Descrizione: Cerca la terapia ATTIVA specifica tramite il nome del farmaco. Restituisce la tupla a 6 elementi se trovata.

Firma: `getTerapieByName(self, farmaco: str) -> tuple`

■ + `getTerapieComplete`

Descrizione: Restituisce TUTTE le terapie (attive e passate) per lo storico.

Firma: `getTerapieComplete(self) -> list`

■ Modulo: `assunzioni.py`

Percorso: ``ui/assunzioni.py``

Nessuna classe pubblica definita in questo modulo.

■ Modulo: `dashboard_medico.py`

Percorso: ``ui/dashboard_medico.py``

Nessuna classe pubblica definita in questo modulo.

■ Modulo: `dashboard_paziente.py`

Percorso: ``ui/dashboard_paziente.py``

Nessuna classe pubblica definita in questo modulo.

■ Modulo: `dettaglio_paziente.py`

Percorso: ``ui/dettaglio_paziente.py``

Nessuna classe pubblica definita in questo modulo.

■ Modulo: `log_page.py`

Percorso: ``ui/log_page.py``

Nessuna classe pubblica definita in questo modulo.

■ Modulo: `registrazione.py`

Percorso: ``ui/registrazione.py``

Nessuna classe pubblica definita in questo modulo.

■ Modulo: sintomi.py

Percorso: `ui/sintomi.py`

Nessuna classe pubblica definita in questo modulo.

■ Modulo: conftest.py

Percorso: `tests/conftest.py`

Nessuna classe pubblica definita in questo modulo.

■ Modulo: test_user.py

Percorso: `tests/test\_user.py`

■■ Classe: TestUser

Descrizione: Nessuna descrizione disponibile.

■ **Attributi** (Variabili di istanza):

Nome	Tipo suggerito	Descrizione
args	Any	
kwargs	Any	

■ Costruttore:

■ Metodi (UML: + pubblico, - privato):

■ + test_get_user_info

Firma: test_get_user_info(self, db_path, init_db)

■ + test_hash_password

Firma: test_hash_password(self)

■ + test_hash_password_diversi

Descrizione: Test che password diverse producano hash diversi

Firma: test_hash_password_diversi(self)

■ + test_is_medico

Firma: test_is_medico(self, db_path, init_db)

■ + test_login_case_sensitive_username

Descrizione: Test che l'username sia case-sensitive

Firma: test_login_case_sensitive_username(self, db_path, init_db)

■ + test_login_con_spazi

Descrizione: Test login con spazi nella password

Firma: test_login_con_spazi(self, db_path, init_db)

■ + test_login_db_non_esistente

Descrizione: Test connessione a database inesistente

Firma: test_login_db_non_esistente(self)

■ + test_login_password_sbagliata

Firma: test_login_password_sbagliata(self, db_path, init_db)

■ + test_login_successo

Firma: test_login_successo(self, db_path, init_db)

■ + test_login_username_inesistente

Firma: test_login_username_inesistente(self, db_path, init_db)

■ + test_password_vuota

Descrizione: Test login con password vuota

Firma: test_password_vuota(self, db_path, init_db)

■ + test_str_medico

Firma: test_str_medico(self, db_path, init_db)

■ + test_str_paziente

Firma: test_str_paziente(self, db_path, init_db)

■ + test_username_vuoto

Descrizione: Test login con username vuoto

Firma: test_username_vuoto(self, db_path, init_db)

■ Modulo: test_paziente.py

Percorso: `tests/test\_paziente.py`

■■ Classe: TestPaziente

Descrizione: Nessuna descrizione disponibile.

■ Attributi (Variabili di istanza):

Nome	Tipo suggerito	Descrizione
args	Any	
kwargs	Any	

■ Costruttore:

■ Metodi (UML: + pubblico, - privato):

■ + test_aggiorna_annotazione

Firma: test_aggiorna_annotazione(self, db_path, init_db)

■ + test_aggiorna_assunzione

Firma: test_aggiorna_assunzione(self, db_path, init_db)

■ + test_aggiorna_rilevazione

Firma: test_aggiorna_rilevazione(self, db_path, init_db)

■ + test_aggiorna_segna1azione

Firma: test_aggiorna_segna1azione(self, db_path, init_db)

■ + test_aggiungi_assunzione

Firma: test_aggiungi_assunzione(self, db_path, init_db)

■ + test_aggiungi_assunzione_senza_terapia

Descrizione: Test aggiunta assunzione senza terapia associata

Firma: test_aggiungi_assunzione_senza_terapia(self, db_path, init_db)

■ + test_aggiungi_rilevazione_glicemia_molto_alta

Descrizione: Test rilevazione con glicemia molto alta

Firma: test_aggiungi_rilevazione_glicemia_molto_alta(self, db_path, init_db)

■ + test_aggiungi_rilevazione_glicemia_negativa

Descrizione: Test che venga sollevata eccezione per glicemia negativa

Firma: test_aggiungi_rilevazione_glicemia_negativa(self, db_path, init_db)

■ + test_aggiungi_rilevazione_glicemia_zero

Descrizione: Test rilevazione con glicemia zero

Firma: test_aggiungi_rilevazione_glicemia_zero(self, db_path, init_db)

■ + test_aggiungi_segna1azione_con_terapia

Firma: test_aggiungi_segna1azione_con_terapia(self, db_path, init_db)

■ + test_annotazione_molto_lunga

Descrizione: Test annotazione con testo molto lungo

Firma: test_annotazione_molto_lunga(self, db_path, init_db)

■ + test_annotazione_null_iniziale

Descrizione: Test che annotazione None venga convertita in stringa vuota

Firma: test_annotazione_null_iniziale(self, db_path, init_db)

■ + test_annotazione_vuota

Descrizione: Test aggiornamento con annotazione vuota

Firma: test_annotazione_vuota(self, db_path, init_db)

■ + test_carica_dati_anagrafici

Firma: test_carica_dati_anagrafici(self, db_path, init_db)

■ + test_elimina_assunzione

Firma: test_elimina_assunzione(self, db_path, init_db)

■ + test_elimina_assunzione_inesistente

Descrizione: Test eliminazione assunzione inesistente

Firma: test_elimina_assunzione_inesistente(self, db_path, init_db)

■ + test_elimina_rilevazione

Firma: test_elimina_rilevazione(self, db_path, init_db)

■ + test_elimina_rilevazione_inesistente

Descrizione: Test eliminazione rilevazione inesistente (non deve crashare)

Firma: test_elimina_rilevazione_inesistente(self, db_path, init_db)

■ + test_elimina_segnalazione

Firma: test_elimina_segnalazione(self, db_path, init_db)

■ + test_elimina_segnalazione_inesistente

Descrizione: Test eliminazione segnalazione inesistente

Firma: test_elimina_segnalazione_inesistente(self, db_path, init_db)

■ + test_get_assunzioni_paziente_senza_assunzioni

Descrizione: Test getAssunzioni su paziente senza assunzioni

Firma: test_get_assunzioni_paziente_senza_assunzioni(self, db_path, init_db)

■ + test_get_medico_riferimento

Firma: test_get_medico_riferimento(self, db_path, init_db)

■ + test_get_rilevazioni

Firma: test_get_rilevazioni(self, db_path, init_db)

■ + test_get_rilevazioni_paziente_senza_rilevazioni

Descrizione: Test getRilevazioni su paziente senza rilevazioni

Firma: test_get_rilevazioni_paziente_senza_rilevazioni(self, db_path, init_db)

■ + test_get_segnalazioni_paziente_senza_segnalazioni

Descrizione: Test getSegnalazioni su paziente senza segnalazioni

Firma: test_get_segnalazioni_paziente_senza_segnalazioni(self, db_path, init_db)

■ + test_get_terapie_by_name

Firma: test_get_terapie_by_name(self, db_path, init_db)

■ + test_get_terapie_complete

Firma: test_get_terapie_complete(self, db_path, init_db)

■ + test_get_terapie_paziente_senza_terapie

Descrizione: Test getTerapie su paziente senza terapie

Firma: test_get_terapie_paziente_senza_terapie(self, db_path, init_db)

■ + test_get_terapie_senza_fine

Firma: test_get_terapie_senza_fine(self, db_path, init_db)

■ + test_getters

Firma: test_getters(self, db_path, init_db)

■ + test_ora_formato_strano

Descrizione: Test rilevazione con formato ora non standard

Firma: test_ora_formato_strano(self, db_path, init_db)

■ + test_paziente_inesistente

Descrizione: Test creazione paziente con id inesistente

Firma: test_paziente_inesistente(self, db_path, init_db)

■ + test_rilevazione_ordine

Descrizione: Test che le rilevazioni siano ordinate per giorno decrescente

Firma: test_rilevazione_ordine(self, db_path, init_db)

■ + test_str

Firma: test_str(self, db_path, init_db)

■ Modulo: test_gravita.py

Percorso: `tests/test\_gravita.py`

■■ Classe: TestGravita

Descrizione: Nessuna descrizione disponibile.

■ Attributi (Variabili di istanza):

Nome	Tipo suggerito	Descrizione
args	Any	
kwargs	Any	

■ Costruttore:

■ Metodi (UML: + pubblico, - privato):

■ + test_gravita_aumenta_con_glicemia_fuori_soglia

Firma: test_gravita_aumenta_con_glicemia_fuori_soglia(self, db_path, init_db)

■ + test_gravita_aumenta_con_terapia_non_rispettata

Firma: test_gravita_aumenta_con_terapia_non_rispettata(self, db_path, init_db)

■ + test_gravita_combinata

Firma: test_gravita_combinata(self, db_path, init_db)

■ + test_gravita_con_assunzione_parziale

Firma: test_gravita_con_assunzione_parziale(self, db_path, init_db)

■ + test_gravita_giorno_senza_rilevazioni

Descrizione: Test gravità per giorno senza rilevazioni

Firma: `test_gravita_giorno_senza_rilevazioni(self, db_path, init_db)`

■ + `test_gravita_glicemia_post_prandiale_alta`

Firma: `test_gravita_glicemia_post_prandiale_alta(self, db_path, init_db)`

■ + `test_gravita_glicemia_post_prandiale_limite`

Descrizione: Test glicemia post-prandiale esattamente a 180 (limite)

Firma: `test_gravita_glicemia_post_prandiale_limite(self, db_path, init_db)`

■ + `test_gravita_glicemia_pre_prandiale_bassa`

Firma: `test_gravita_glicemia_pre_prandiale_bassa(self, db_path, init_db)`

■ + `test_gravita_glicemia_pre_prandiale_limite_inferiore`

Descrizione: Test glicemia pre-prandiale esattamente a 80 (limite)

Firma: `test_gravita_glicemia_pre_prandiale_limite_inferiore(self, db_path, init_db)`

■ + `test_gravita_glicemia_pre_prandiale_limite_superiore`

Descrizione: Test glicemia pre-prandiale esattamente a 130 (limite)

Firma: `test_gravita_glicemia_pre_prandiale_limite_superiore(self, db_path, init_db)`

■ + `test_gravita_multiple_rilevazioni`

Firma: `test_gravita_multiple_rilevazioni(self, db_path, init_db)`

■ + `test_gravita_piu_giorni`

Descrizione: Test gravità con rilevazioni in giorni diversi

Firma: `test_gravita_piu_giorni(self, db_path, init_db)`

■ + `test_gravita_solo_assunzioni_complete`

Descrizione: Test gravità zero quando tutte le assunzioni sono fatte

Firma: `test_gravita_solo_assunzioni_complete(self, db_path, init_db)`

■ + `test_gravita_terapia_con_data_fine_futura_e_attiva`

Firma: `test_gravita_terapia_con_data_fine_futura_e_attiva(self, db_path, init_db)`

■ + `test_gravita_zero_con_glicemia_ok`

Firma: `test_gravita_zero_con_glicemia_ok(self, db_path, init_db)`

■ Modulo: `test_medico.py`

Percorso: ``tests/test_medico.py``

■■ Classe: `TestMedico`

Descrizione: Nessuna descrizione disponibile.

■ **Attributi** (Variabili di istanza):

Nome	Tipo suggerito	Descrizione
args	Any	
kwargs	Any	

■ Costruttore:

■ Metodi (UML: + pubblico, - privato):

■ + test_elimina_terapia_definitiva

Firma: test_elimina_terapia_definitiva(self, db_path, init_db)

■ + test_elimina_terapia_inesistente

Descrizione: Test eliminazione terapia inesistente

Firma: test_elimina_terapia_inesistente(self, db_path, init_db)

■ + test_get_log_operazioni_limite

Firma: test_get_log_operazioni_limite(self, db_path, init_db)

■ + test_get_paziente_by_id

Firma: test_get_paziente_by_id(self, db_path, init_db)

■ + test_get_pazienti

Firma: test_get_pazienti(self, db_path, init_db)

■ + test_get_pazienti_con_dettaglio

Firma: test_get_pazienti_con_dettaglio(self, db_path, init_db)

■ + test_get_pazienti_ids

Firma: test_get_pazienti_ids(self, db_path, init_db)

■ + test_get_statistiche

Firma: test_get_statistiche(self, db_path, init_db)

■ + test_getters

Firma: test_getters(self, db_path, init_db)

■ + test_getters_medico

Descrizione: Test getters medico con tutti i campi

Firma: test_getters_medico(self, db_path, init_db)

■ + test_interrompi_terapia

Firma: test_interrompi_terapia(self, db_path, init_db)

■ + test_interrompi_terapia_inesistente

Descrizione: Test interruzione terapia inesistente

Firma: test_interrompi_terapia_inesistente(self, db_path, init_db)

■ + test_modifica_terapia_farmaco_diverso

Firma: test_modifica_terapia_farmaco_diverso(self, db_path, init_db)

■ + test_modifica_terapia_stesso_farmaco

Firma: test_modifica_terapia_stesso_farmaco(self, db_path, init_db)

■ + test_paziente_senza_medico

Descrizione: Test caricamento medico senza pazienti

Firma: test_paziente_senza_medico(self, db_path, init_db)

■ + test_prescrivi_terapia

Firma: test_prescrivi_terapia(self, db_path, init_db)

■ + test_prescrivi_terapia_con_data_fine

Descrizione: Test prescrizione con data_fine esplicita

Firma: test_prescrivi_terapia_con_data_fine(self, db_path, init_db)

■ + test_prescrivi_terapia_paziente_non_assegnato

Descrizione: Test prescrizione a paziente non assegnato al medico

Firma: test_prescrivi_terapia_paziente_non_assegnato(self, db_path, init_db)

■ + test_prescrivi_terapia_quantita_zero

Descrizione: Test prescrizione con quantità zero - non deve essere inserita

Firma: test_prescrivi_terapia_quantita_zero(self, db_path, init_db)

■ + test_registra_operazione

Firma: test_registra_operazione(self, db_path, init_db)

■ + test_str_medico

Firma: test_str_medico(self, db_path, init_db)

■ Modulo: test_consistency.py

Percorso: `tests/test\_consistency.py`

■■ Classe: TestConsistency

Descrizione: Nessuna descrizione disponibile.

■ Attributi (Variabili di istanza):

Nome	Tipo suggerito	Descrizione
args	Any	
kwargs	Any	

■ Costruttore:

■ Metodi (UML: + pubblico, - privato):

■ + test_cf_duplicato_anagrafica

Descrizione: Test che non si possa inserire lo stesso CF due volte in ANAGRAFICA

Firma: test_cf_duplicato_anagrafica(self, db_path, init_db)

■ + test_data_fine_anteriore_data_inizio

Descrizione: Test che data_fine non possa essere anteriore a data_inizio

Firma: test_data_fine_anteriore_data_inizio(self, db_path, init_db)

■ + test_ordinamento_rilevazioni_dopo_aggiornamento

Descrizione: Test che le rilevazioni rimangano ordinate dopo un aggiornamento

Firma: test_ordinamento_rilevazioni_dopo_aggiornamento(self, db_path, init_db)

■ + test_username_duplicato_user

Descrizione: Test che non si possa inserire lo stesso username due volte in USER

Firma: test_username_duplicato_user(self, db_path, init_db)

■ Modulo: test_interaction.py

Percorso: `tests/test\_interaction.py`

■■ Classe: TestInteraction

Descrizione: Nessuna descrizione disponibile.

■ Attributi (Variabili di istanza):

Nome	Tipo suggerito	Descrizione
args	Any	
kwargs	Any	

■ Costruttore:

■ Metodi (UML: + pubblico, - privato):

■ + test_flusso_completo_prescrizione_assunzione

Descrizione: Test end-to-end: medico prescrive -> paziente assume -> medico verifica

Firma: test_flusso_completo_prescrizione_assunzione(self, db_path, init_db)

■ + test_interazione_due_medici_stesso_paziente

Descrizione: Due medici visualizzano lo stesso paziente

Firma: test_interazione_due_medici_stesso_paziente(self, db_path, init_db)

■ + test_operazione_genera_log

Descrizione: Verifica che ogni operazione del medico generi un log

Firma: test_operazione_genera_log(self, db_path, init_db)

--- Fine Specifiche UML ---